UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

Escuela Profesional de Ingeniería Informática y de Sistemas

CIRCUITOS CUÁNTICOS REVERSIBLES PARA FUNCIONES BOOLEANAS: REVISIÓN Y VALIDACIÓN EN QISKIT

REVERSIBLE QUANTUM CIRCUITS FOR BOOLEAN FUNCTIONS: A REVIEW AND VALIDATION IN QISKIT

Brayan Vladymir Molocho Condori[1] **Haider Alex Carpio Hermoza**[2]
Efraín Vitorino Marín[3]

Códigos: [1]164244; [2]164563; [3]160337.
Universidad Nacional de San Antonio Abad del Cusco, Cusco, Perú.
Asignatura: Computación Cuántica (IF460AIN). Docente: Raúl Huillca Huallparimachi.
Semestre 2026-II · Avance del proyecto · Noviembre de 2026.

Resumen— La implementación cuántica de funciones booleanas exige preservar la información mediante transformaciones reversibles. Este avance integra una revisión dirigida de cuatro trabajos de 2024–2026 con una implementación reproducible en Qiskit. Se construyen AND, NAND, OR, NOR y XOR a partir de un semisumador, usando una realización con Toffoli y otra con la transformada cuántica de Fourier (QFT). Las dos familias se verifican con 40 casos de tablas de verdad y con la comparación de sus operadores completos. Además, se validan 552 casos de conteo de población y AND con 2, 4 y 8 entradas, y se compilan circuitos de hasta 16 entradas. Todas las validaciones ideales satisfacen una tolerancia de 10^{-10}. Para 16 entradas, el conteo mediante QFT emplea 20 qubits frente a 31 de la escalera de Toffoli, pero requiere 175 compuertas CX frente a 90. Los resultados evidencian un compromiso entre ancho, compuertas y profundidad; la simulación ideal realizada no permite concluir superioridad en hardware.

Palabras clave: circuitos reversibles, funciones booleanas, QFT, Toffoli, Qiskit, recursos cuánticos.

Abstract— Boolean functions must preserve information when implemented as reversible quantum transformations. This progress report combines a focused review of four papers published in 2024–2026 with a reproducible Qiskit implementation. AND, NAND, OR, NOR and XOR are constructed from a common half-adder template, using either Toffoli gates or the quantum Fourier transform (QFT). Both families pass 40 truth-table cases and complete operator comparisons. An additional 552 population-count and AND cases are checked for 2, 4 and 8 inputs, while circuits with up to 16 inputs are compiled. All ideal checks meet a tolerance of 10^{-10}. At 16 inputs, the QFT circuit requires 20 qubits rather than the Toffoli ladder's 31, but uses 175 CX gates instead of 90. These results expose a trade-off between width, gate count and depth; ideal simulation alone does not establish a hardware advantage.

Keywords: reversible circuits, Boolean functions, QFT, Toffoli, Qiskit, quantum resources.

I. INTRODUCCIÓN

Las compuertas booleanas AND y OR convierten dos bits en uno; por sí solas no permiten recuperar la entrada. Un circuito cuántico cerrado, en cambio, actúa mediante un operador unitario y conserva la información. Por ello, implementar lógica clásica en computación cuántica requiere incluir registros auxiliares o salidas residuales que vuelvan reversible la transformación [1].

La revisión presentada en la primera parte del proyecto identificó cuatro enfoques: aritmética basada en QFT, composición de circuitos digitales, síntesis mediante tablas de búsqueda y minimización visual [2–5]. Este avance mantiene ese tema y añade una comprobación experimental propia. La pregunta que orienta el trabajo es: *¿cómo cambian los recursos al realizar funciones booleanas reversibles con QFT en lugar de una construcción basada en Toffoli?*

La comparación exige distinguir el número de qubits de la cantidad de compuertas y de la profundidad. Un circuito más estrecho puede necesitar más interacciones; además, un resultado correcto en simulación sin ruido no es una estimación de fidelidad física. El aporte del avance consiste en hacer explícitas estas condiciones y enlazar el análisis bibliográfico con código ejecutable, datos tabulares y figuras generadas desde una misma ejecución.

II. OBJETIVOS

Objetivo general. Analizar e implementar circuitos cuánticos reversibles para funciones booleanas, contrastando diseños basados en QFT y en Toffoli bajo condiciones comunes de compilación.

Objetivos específicos. Fundamentar la reversibilidad con notación de Dirac y matrices unitarias; organizar los cuatro antecedentes de la primera entrega; construir y validar cinco compuertas de dos entradas; estudiar el escalamiento de un contador QFT y una escalera AND; y documentar los recursos, las restricciones y los pasos necesarios para reproducir los experimentos.

III. FUNDAMENTOS TEÓRICOS

A. Estados, producto interno y producto tensorial

Un qubit puro se escribe como

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \qquad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (1)$$

Su normalización se expresa con el producto interno $\langle\psi|\psi\rangle = 1$. Los estados de la base computacional son ortogonales: $\langle 0|1\rangle = 0$. Un sistema de n qubits pertenece a $(\mathbb{C}^2)^{\otimes n}$ y tiene 2^n amplitudes; el producto tensorial permite construir estados como $|a\rangle \otimes |b\rangle = |ab\rangle$. La medición de una cadena x devuelve esa cadena con probabilidad $|\langle x|\psi\rangle|^2$ [1].

Una matriz estocástica describe transiciones de probabilidades clásicas. Una matriz unitaria transforma amplitudes complejas y satisface $U^\dagger U = I$. Aunque $|U_{yx}|^2$ define probabilidades de transición para entradas de la base, reemplazar las amplitudes por probabilidades elimina las fases necesarias para describir interferencia y superposición.

B. Compuertas y reversibilidad

Las puertas elementales utilizadas incluyen

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \quad (2)$$

CNOT transforma (a, b) en $(a, b \oplus a)$ y Toffoli transforma (a, b, c) en $(a, b, c \oplus ab)$. Una función clásica f admite la inclusión reversible $U_f|x\rangle|z\rangle = |x\rangle|z \oplus f(x)\rangle$. También puede implementarse mediante otra permutación reversible que deje $f(x)$ en una línea identificada y conserve información residual en las demás. La familia de semisumadores de este proyecto utiliza esta segunda opción.

C. Transformada cuántica de Fourier

Para $M = 2^m$, la QFT se define como

$$F_M|t\rangle = \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} e^{2\pi i t k/M}|k\rangle. \quad (3)$$

La suma puede realizarse mediante fases en la base de Fourier [6]. Si $D_b|k\rangle = e^{2\pi i b k/M}|k\rangle$,

$$F_M^\dagger D_b F_M|t\rangle = |t + b \bmod M\rangle. \quad (4)$$

Para $M = 4$, se necesitan dos qubits en el acumulador y un control b. En Qiskit se emplea `QFTGate`, con la convención de que el qubit de índice menor tiene menor peso binario [7]. La QFT también conecta con estimación de fase y con la búsqueda de períodos de Shor; aquí se estudia su uso en suma reversible.

IV. ANTECEDENTES Y REVISIÓN DIRIGIDA

A. Diseño lógico mediante QFT

Kaltehei et al. [2] construyen una familia de compuertas con un semisumador QFT y operaciones X/SWAP. Extienden el diseño a conteo de población, del que obtienen AND y paridad. Su aporte para este proyecto es mostrar que una plantilla aritmética permite generar varias funciones y reducir el ancho frente a una escalera con muchos auxiliares. El beneficio en qubits debe evaluarse junto con las compuertas de dos qubits y la profundidad. La implementación propia adopta esa arquitectura, con una convención de registros explícita, y obtiene sus métricas nuevamente.

B. Composición de circuitos digitales

Escanez-Exposito y Caballero-Gil [3] presentan QuantumSolver Composer, una herramienta basada en Qiskit para expresar circuitos digitales y esquemas criptográficos mediante circuitos cuánticos. El diseño modular facilita el aprendizaje de sumadores y de otros bloques lógicos. Su utilidad didáctica no equivale a una optimización automática de todos los recursos. Este antecedente motiva separar en el código la construcción de circuitos, su verificación y la generación de resultados. No se reproducen aquí sus cifradores ni las operaciones de reinicio empleadas en algunas construcciones.

C. Síntesis LUT y fases relativas

Clarino et al. [4] explotan pares de cómputo y descomputación en redes basadas en tablas de búsqueda (LUT). Las descomposiciones de Shannon y Davio permiten implementar funciones con fases relativas y reducir el conteo de puertas T cuando esas fases se cancelan en el circuito completo. El resultado depende del contexto de uso y de la estructura de la red. Para este avance, la principal consecuencia metodológica es que verificar solo una tabla de verdad no basta: dos circuitos pueden dar los mismos bits y diferir en fases que afectan entradas en superposición.

D. Síntesis visual Q-Map

Hajjdiab et al. [5] proponen Q-Map como una técnica visual de realización cuántica de funciones booleanas. El uso de relaciones de álgebra booleana facilita obtener construcciones compactas para los ejemplos considerados. El enfoque ofrece una perspectiva complementaria a las plantillas aritméticas, aunque no sustituye una comparación de compilaciones equivalentes. En este proyecto se usa como antecedente de minimización; no se presenta una implementación propia del algoritmo Q-Map.

V. METODOLOGÍA E IMPLEMENTACIÓN

A. Alcance y fuentes

Se desarrolla un estudio comparativo de simulación ideal y compilación. La revisión es dirigida: parte de los cuatro artículos de la entrega anterior y contrasta sus datos editoriales con las fuentes originales. Se complementa con fundamentos de computación cuántica y documentación oficial de Qiskit. No se atribuyen al proceso búsquedas exhaustivas ni cantidades de selección de estudios propias de una revisión sistemática no realizada.

Se consideran tres niveles: funciones de dos entradas, equivalencia de operadores y escalamiento a $N = 2, 4, 8, 16$. Las tablas de verdad se generan localmente, por lo que no se requiere descargar un conjunto de datos externo. La unidad de observación es una entrada binaria aplicada a un circuito.

Tabla 1: Síntesis de los cuatro antecedentes conservados de la primera entrega.

Trabajo	Estrategia	Recursos de interés	Implicación para este avance
Kaltehei et al. [2]	Semisumador y contador mediante QFT	Qubits, CX y profundidad	Comparar el ahorro de ancho con el aumento de interacciones.
Escanez-Exposito y Caballero-Gil [3]	Composición modular de circuitos	Costo de bloques y auxiliares	Mantener circuitos y verificaciones separados y reutilizables.
Clarino et al. [4]	Síntesis LUT con fases relativas	Conteo T y cómputo/descomputación	Verificar fases; el conteo CX no reemplaza al conteo T.
Hajjdiab et al. [5]	Minimización visual Q-Map	Compuertas y auxiliares	Distinguir ejemplos compactos de optimalidad general.

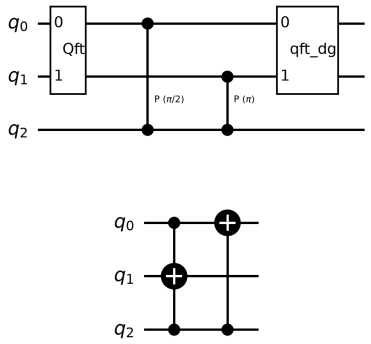


Figura 1: Semisumadores implementados: QFT (arriba) y Toffoli/CNOT (abajo). Las figuras muestran los circuitos antes de transpilar.

B. Semisumadores equivalentes

Se fijan $q_0 = a$, $q_1 = c$ y $q_2 = b$, donde c inicia en cero en las tablas. El acumulador representa $t = a + 2c$. El diseño QFT implementa

$$(t, b) \longmapsto (t + b \bmod 4, b). \tag{5}$$

Con $c = 0$, las salidas son $q_0 = a \oplus b$ y $q_1 = ab$. La implementación alternativa aplica Toffoli con controles q_0, q_2 sobre q_1, seguida de CNOT de q_2 a q_0. Ambas implementan la misma permutación para los ocho estados de la base, incluidos los de $c = 1$.

Listing 1: Núcleo del semisumador QFT implementado.

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import QFTGate
from numpy import pi

qc = QuantumCircuit(3)
qc.append(QFTGate(2), [0, 1])
qc.cp(pi / 2, 2, 0)
qc.cp(pi, 2, 1)
qc.append(QFTGate(2).inverse(), [0, 1])
```

Para AND se lee q_1; NAND añade X sobre esa salida. NOR invierte ambas entradas antes del sumador; OR añade además X en la salida. XOR intercambia q_0 y q_1 al finalizar. Estas transformaciones se aplican a ambos diseños para comparar la misma operación completa. Las otras líneas retienen información residual; no se presupone que ambas entradas permanezcan intactas.

Tabla 2: Tabla de verdad de referencia; en los circuitos q_1 es la salida.

a	b	AND	NAND	OR	NOR	XOR
0	0	0	1	0	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	0	1	0	0

C. Conteo de población y escalera AND

Para $N = 2^k$, el contador QFT reutiliza la primera entrada como bit menos significativo de un acumulador de $k + 1$ qubits. Añade k auxiliares y mantiene las $N - 1$ entradas restantes como controles. En la base de Fourier, cada control suma una unidad mediante fases controladas; la QFT inversa devuelve el peso de Hamming $s = \sum_i x_i$. Así, el ancho total es $N + k$ y las rotaciones controladas de las adiciones son $(N - 1)(k + 1)$, además de las transformadas.

El bit de mayor peso vale uno exactamente cuando $s = N$, y por tanto representa AND; el de menor peso representa la paridad. La escalera alternativa usa $N - 1$ Toffoli y $N - 1$ auxiliares, para un ancho de $2N - 1$ qubits. Conserva las entradas y productos intermedios sin descomputarlos. Ambos diseños proporcionan AND, pero sus registros residuales y sus capacidades adicionales son distintos: el contador también entrega s. Por ello, en este experimento se compara la tarea AND y los recursos, sin afirmar equivalencia de los operadores completos de ambos diseños multientrada.

D. Protocolo de validación

Se prueban las cuatro entradas de cada compuerta en cada familia: cinco funciones por cuatro entradas por dos diseños, es decir, 40 casos. La simulación utiliza `Statevector` [8] y verifica la probabilidad exacta del resultado esperado. Para las cinco parejas de circuitos de tres qubits se calcula

$$\varepsilon = \max_{ij} |(U_{\mathrm{QFT}} - U_{\mathrm{Toffoli}})_{ij}|, \tag{6}$$

además de $\max_{ij} |(U^\dagger U - I)_{ij}|$. Se adopta tolerancia 10^{-10}. Esta comparación incluye amplitudes y fases de los ocho estados de la base, por lo que es más exigente que comprobar únicamente probabilidades.

Para cada diseño multientrada se exploran exhaustivamente 2^N entradas cuando $N = 2, 4, 8$: $2(4+16+256) = 552$ casos. En QFT se comprueba el conteo completo; en la escalera, AND. Para $N = 16$ se construyen y compilan los circuitos,

Tabla 3: Recursos propios para las cinco compuertas de dos entradas. Todos los circuitos usan tres qubits.

Función	Diseño	CX	Prof.	Total
AND	Toffoli	5	11	14
AND	QFT	9	19	29
NAND	Toffoli	5	11	14
NAND	QFT	9	19	29
OR	Toffoli	5	11	16
OR	QFT	9	19	29
NOR	Toffoli	5	11	16
NOR	QFT	9	19	29
XOR	Toffoli	5	11	14
XOR	QFT	9	19	29

sin simular sus 65 536 entradas. Una demostración adicional prepara H en q_0 y q_2, aplica el semisumador QFT y obtiene 4096 muestras con semilla 42. El histograma es muestreo ideal en CPU.

E. Compilación y reproducibilidad

Se usa Python 3.12 y Qiskit 2.6.2, con dependencias fijadas. Ambas alternativas se transpilan a $\{U, \mathrm{CX}\}$, con nivel de optimización 3, semilla 42 y conectividad completa [9]. Se indica `qubits_initially_zero=False`, de modo que la compilación no suponga que las entradas lógicas son cero. El ancho, el número de CX, la profundidad y el total de compuertas se calculan sobre esos circuitos; la preparación y la medición se excluyen de las tablas de recursos.

El compilador puede representar un SWAP final mediante una permutación de las salidas. Esa permutación se registra y se utiliza al comparar el operador compilado con el original mediante `Operator.from_circuit` [10]. Por tanto, los conteos corresponden a circuitos con etiquetado lógico de salidas, sin exigir que las líneas finales mantengan un orden físico fijo. La exportación OpenQASM incluye mediciones.

VI. RESULTADOS DEL AVANCE

A. Corrección y coherencia

Los 40 casos de tablas de verdad coinciden con las funciones esperadas. La probabilidad mínima se redondea a 1.000000. La discrepancia máxima entre las parejas de operadores y entre las versiones compiladas y originales es `3.70e-15`, inferior a la tolerancia establecida. La prueba de unitariedad también satisface esa tolerancia. Estas verificaciones respaldan la corrección de los circuitos ideales de tres qubits, incluidas entradas coherentes.

La Tabla 3 muestra que la implementación QFT de esta plantilla requiere más CX y mayor profundidad. Estos valores son resultados de la versión y las condiciones indicadas; no se copian de los artículos. La igualdad de recursos entre algunas funciones proviene de las simplificaciones del compilador y del tratamiento de las permutaciones de salida. En particular, la fila XOR evalúa la plantilla de semisumador completa; un XOR que únicamente conserva las dos entradas y actualiza un objetivo puede construirse con dos CNOT. No se atribuye optimalidad a la plantilla.

La Tabla 4 recoge 552 verificaciones adicionales satisfactorias. En el experimento con entradas uniformes en superposición, medir el acumulador produce probabilidades teóricas $P(00) = 1/4$, $P(01) = 1/2$ y $P(10) = 1/4$: corresponden a sumar dos bits uniformes. La Figura 2 contrasta esas probabilidades con las muestras; las pequeñas diferencias provienen del muestreo finito.

Tabla 4: Validación exhaustiva multientrada. Las probabilidades se muestran con seis decimales; las diferencias residuales son de redondeo numérico.

N	Diseño	Casos	Prob. mínima
2	QFT	4	1.000000
2	Toffoli	4	1.000000
4	QFT	16	1.000000
4	Toffoli	16	1.000000
8	QFT	256	1.000000
8	Toffoli	256	1.000000

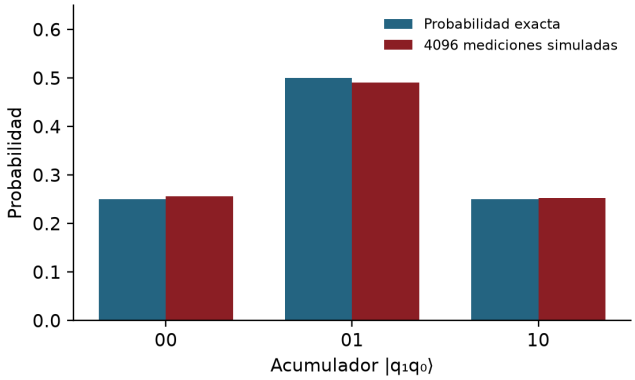


Figura 2: Medición simulada del acumulador tras aplicar el semisumador QFT a entradas en superposición. No se usó un procesador cuántico físico.

B. Escalamiento de recursos

La Tabla 5 y la Figura 3 muestran el compromiso principal. Para $N = 16$, el contador usa $16 + \log_2 16 = 20$ qubits y la escalera usa $2(16) - 1 = 31$: la reducción es de 11 qubits, aproximadamente 35,5 %. En cambio, los conteos son 175 y 90 CX, respectivamente, y las profundidades son 118 y 95. El contador reduce ancho a costa de más interacciones y capas. Las cifras de $N = 16$ describen compilación; la validación exhaustiva propia se limita a $N \leq 8$.

Tabla 5: Recursos compilados propios para AND multientrada. QFT entrega además el peso de Hamming; Toffoli corresponde a la escalera sin descomputación.

N	Diseño	Qubits	CX	Prof.
2	QFT	3	9	19
2	Toffoli	3	6	11
4	QFT	6	27	38
4	Toffoli	7	18	23
8	QFT	11	73	70
8	Toffoli	15	42	47
16	QFT	20	175	118
16	Toffoli	31	90	95

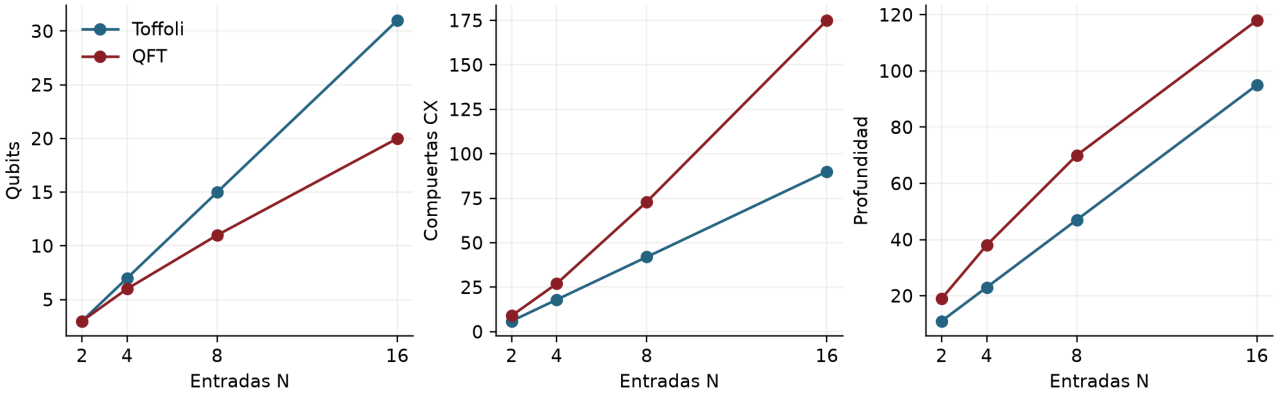


Figura 3: Comparación propia de ancho, CX y profundidad bajo una misma base y conectividad completa. Las líneas unen los tamaños compilados; no representan mediciones de tiempo ni de fidelidad en hardware.

VII. DISCUSIÓN Y LIMITACIONES

Los resultados concuerdan con el compromiso de recursos que motiva el diseño QFT de la literatura: ahorrar auxiliares no garantiza reducir compuertas. La generalización del contador añade $(N-1)(k+1)$ fases controladas y dos transformadas de $k+1$ qubits. La escalera añade $N-1$ Toffoli y conserva productos parciales en un registro mayor. La elección depende de la disponibilidad de qubits y del costo físico de las interacciones.

La revisión y la experimentación cumplen funciones distintas. Los artículos LUT y Q-Map aportan perspectivas de síntesis; las pruebas propias solo implementan la familia de semisumadores y las dos construcciones de escalamiento descritas. No se reproducen las ejecuciones en hardware, los cifradores ni los bancos de pruebas de los cuatro artículos. Asimismo, contar CX en una base de rotaciones continuas no permite inferir directamente un conteo T en un régimen tolerante a fallos.

La simulación exacta no incluye decoherencia, errores de lectura, errores de puertas ni conectividad restringida. Un valor de probabilidad cercano a uno demuestra consistencia matemática del modelo implementado, no fidelidad de una QPU. En una arquitectura con acoplamientos limitados, el enrutamiento puede introducir SWAP y modificar la comparación. Las fases pequeñas del contador también exigen estudiar precisión o aproximaciones antes de extrapolar a tamaños mayores.

La escalera conserva entradas y productos parciales, mientras que el contador reutiliza una entrada y proporciona una suma. La tabla de escalamiento compara la disponibilidad de AND como salida, no interfaces reversibles idénticas. Una integración en un algoritmo mayor deberá especificar qué registros deben conservarse y cuáles deben descomputarse. Exigir esa limpieza puede modificar ancho y profundidad en ambas alternativas.

Por último, los conteos pueden variar al cambiar la versión del compilador, el conjunto de puertas o la política de permutaciones. Las dependencias, la configuración y las permutaciones se guardan junto con los resultados. Las diferencias frente a cifras publicadas deben examinarse bajo esas condiciones, en lugar de asumir que implementaciones distintas son una reproducción literal del mismo experimento.

VIII. CONCLUSIONES Y TRABAJO PENDIENTE

Se transformó el avance bibliográfico sobre circuitos reversibles en un estudio con implementación Qiskit, validación y resultados propios. Las cinco funciones de dos entradas pasan sus 40 casos de tablas de verdad, y la comparación de operadores confirma que las dos realizaciones implementan la misma transformación, incluidas sus fases. El contador y la escalera superan además 552 casos exhaustivos para 2, 4 y 8 entradas.

El análisis de hasta 16 entradas demuestra que el diseño QFT ofrece un ancho menor que la escalera considerada, pero requiere más CX y mayor profundidad en las condiciones utilizadas. Por ello, este avance respalda una evaluación con varias métricas y no una afirmación de ventaja general de QFT.

La siguiente etapa contempla simulación con ruido y conectividad de un dispositivo especificado, comparación con descomposiciones MCX de pocos auxiliares, evaluación de QFT aproximada y descomputación de registros residuales. Cualquier resultado físico se documentará con fecha, dispositivo, calibración y número de mediciones. Esas actividades permanecen pendientes.

IX. MATERIAL REPRODUCIBLE

El código principal es `proyecto/experimento.py`; su ejecución desde la raíz con `python -m proyecto.experimento` genera tablas CSV, métricas JSON, diagramas, un circuito OpenQASM 3 y el archivo de macros `resultados.tex`. El artículo incorpora esas macros y figuras para mantener las cifras sincronizadas. El repositorio incluye instrucciones de instalación y 12 pruebas automatizadas. El paquete de Overleaf contiene `main.tex`, la bibliografía y todos los recursos del documento.

REFERENCIAS

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[2] A. Kaltehei, M. Kurt, A. Gençten, and S. Çakmak, "Resource-efficient quantum circuit design for reversible logic implementation," *Quantum Engineering*, vol. 2026, p. 9882453, 2026. [Online]. Available: https://doi.org/10.1155/que2/9882453

[3] D. Escanez-Exposito and P. Caballero-Gil, "Quantum simulation of Boolean logic, digital circuits, and cryptographic schemes," *Logic Journal of the IGPL*, vol. 34, no. 1, p. jzaf019, 2026. [Online]. Available: https://doi.org/10.1093/jigpal/jzaf019

[4] D. Clarino, N. Asada, A. Matsuo, and S. Yamashita, "Leveraging different Boolean function decompositions to reduce T-Count in LUT-Based quantum circuit synthesis," *IEICE Transactions on Information and Systems*, vol. E108-D, no. 4, pp. 320–329, 2025. [Online]. Available: https://doi.org/10.1587/transinf.2024EDP7194

[5] H. Hajjdiab, A. Khalil, and H. Eleuch, "Q-Map: quantum circuit implementation of boolean functions," *Physica Scripta*, vol. 99, no. 8, p. 085127, 2024. [Online]. Available: https://doi.org/10.1088/1402-4896/ad5c17

[6] T. G. Draper, "Addition on a quantum computer," arXiv:quant-ph/0008033, 2000. [Online]. Available: https://arxiv.org/abs/quant-ph/0008033

[7] IBM Quantum, "QFTGate: referencia de la API de Qiskit," 2026, consulta: 11 de noviembre de 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.QFTGate

[8] ——, "Statevector: referencia de la API de Qiskit," 2026, consulta: 11 de noviembre de 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector

[9] ——, "Compilation routines: qiskit.compiler.transpile," 2026, consulta: 11 de noviembre de 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/en/api/qiskit/compiler

[10] ——, "Operator: referencia de la API de Qiskit," 2026, consulta: 11 de noviembre de 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Operator